

FastAPI in Containers - Docker

When deploying FastAPI applications a common approach is to build a **Linux container image**. It's normally done using **Docker** [🔗](#). You can then deploy that container image in one of a few possible ways.

Using Linux containers has several advantages including **security**, **replicability**, **simplicity**, and others.



Tip

In a hurry and already know this stuff? Jump to the [Dockerfile](#) below 📌.



Dockerfile Preview

```
FROM python:3.14

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

COPY ./app /code/app

CMD ["fastapi", "run", "app/main.py", "--port", "80"]

# If running behind a proxy like Nginx or Traefik add --proxy-headers
# CMD ["fastapi", "run", "app/main.py", "--port", "80", "--proxy-headers"]
```

What is a Container

Containers (mainly Linux containers) are a very **lightweight** way to package applications including all their dependencies and necessary files while keeping them isolated from other containers (other applications or components) in the same system.

Linux containers run using the same Linux kernel of the host (machine, virtual machine, cloud server, etc). This just means that they are very lightweight (compared to full virtual machines emulating an entire operating system).

Ask AI

This way, containers consume **little resources**, an amount comparable to running the processes directly (a virtual machine would consume much more).

Containers also have their own **isolated** running processes (commonly just one process), file system, and network, simplifying deployment, security, development, etc.

What is a Container Image

A **container** is run from a **container image**.

A container image is a **static** version of all the files, environment variables, and the default command/program that should be present in a container. **Static** here means that the container **image** is not running, it's not being executed, it's only the packaged files and metadata.

In contrast to a "**container image**" that is the stored static contents, a "**container**" normally refers to the running instance, the thing that is being **executed**.



When the **container** is started and running (started from a **container image**) it could create or change files, environment variables, etc. Those changes will exist only in that container, but would not persist in the underlying container image (would not be saved to disk).

A container image is comparable to the **program** file and contents, e.g. `python` and some file `main.py`.

And the **container** itself (in contrast to the **container image**) is the actual running instance of the image, comparable to a **process**. In fact, a container is running only when it has a **process running** (and normally it's only a single process). The container stops when there's no process running in it.

Container Images

Docker has been one of the main tools to create and manage **container images** and **containers**.

And there's a public [Docker Hub](#) with pre-made **official container images** for many tools, environments, databases, and applications.

For example, there's an official [Python Image](#).

And there are many other images for different things like databases, for example for:

- [PostgreSQL](#)
- [MySQL](#)
- [MongoDB](#)

- [Redis](#) , etc.

By using a pre-made container image it's very easy to **combine** and use different tools. For example, to try out a new database. In most cases, you can use the **official images**, and just configure them with environment variables.

That way, in many cases you can learn about containers and Docker and reuse that knowledge with many different tools and components.

So, you would run **multiple containers** with different things, like a database, a Python application, a web server with a React frontend application, and connect them together via their internal network.

All the container management systems (like Docker or Kubernetes) have these networking features integrated into them.

Containers and Processes

A **container image** normally includes in its metadata the default program or command that should be run when the **container** is started and the parameters to be passed to that program. Very similar to what would be if it was in the command line.

When a **container** is started, it will run that command/program (although you can override it and make it run a different command/program).

A container is running as long as the **main process** (command or program) is running.

A container normally has a **single process**, but it's also possible to start subprocesses from the main process, and that way you will have **multiple processes** in the same container.

But it's not possible to have a running container without **at least one running process**. If the main process stops, the container stops.

Build a Docker Image for FastAPI

Okay, let's build something now! 

I'll show you how to build a **Docker image** for FastAPI **from scratch**, based on the **official Python** image.

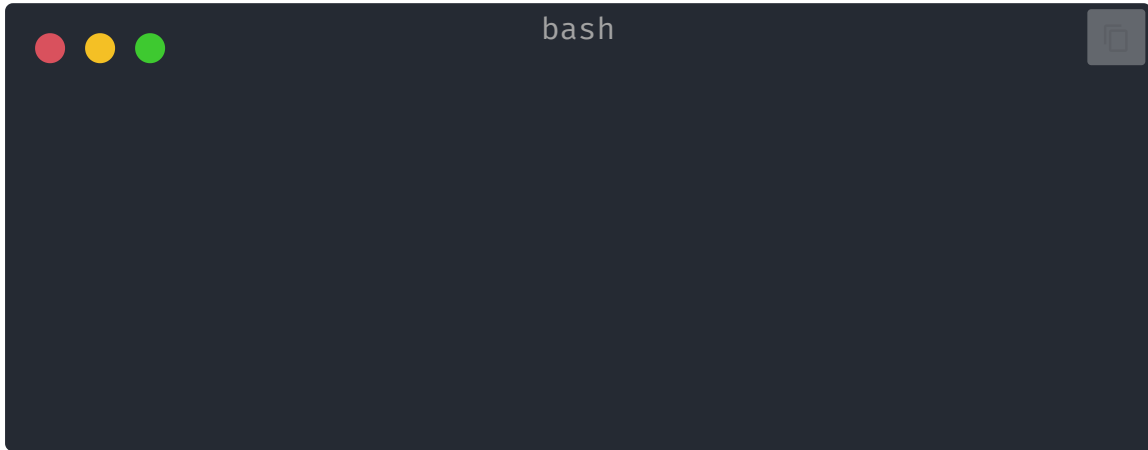
This is what you would want to do in **most cases**, for example:

- Using **Kubernetes** or similar tools
- When running on a **Raspberry Pi**
- Using a cloud service that would run a container image for you, etc.

Package Requirements

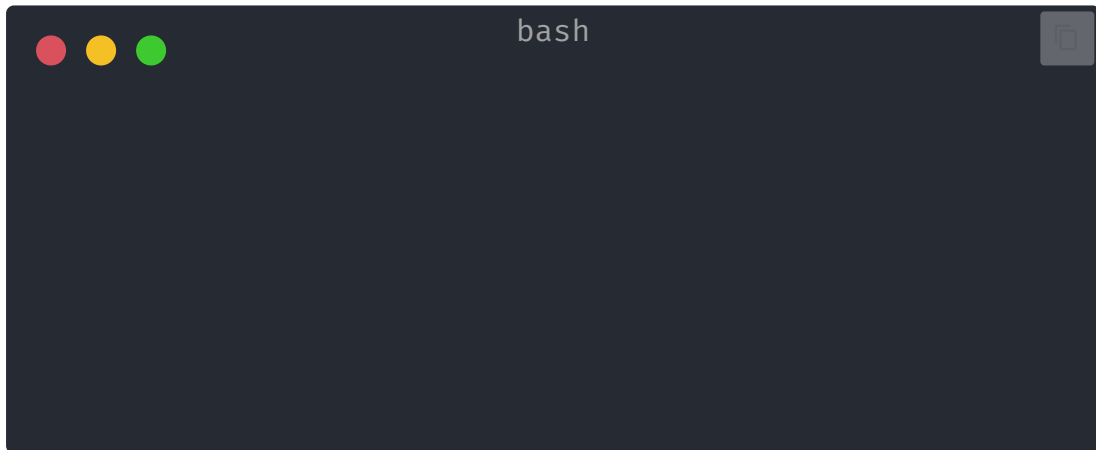
When you manage your project with `uv`, its direct dependencies are declared in `pyproject.toml` and the exact resolved versions are stored in `uv.lock`.

You can add the packages your application needs with:



Note

The Dockerfile below uses `pip` inside the container. You can export the locked dependencies from your `uv` project to the `requirements.txt` format it expects:



The generated `requirements.txt` is an export for the container build. Continue managing dependencies with `uv add` and regenerate it when `uv.lock` changes.

Create the **FastAPI** Code

- Create an `app` directory and enter it.
- Create an empty file `__init__.py`.
- Create a `main.py` file with:

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"Hello": "World"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None):
    return {"item_id": item_id, "q": q}

```

Dockerfile

Now in the same project directory create a file `Dockerfile` with:

```

1 FROM python:3.14

2 WORKDIR /code

3 COPY ./requirements.txt /code/requirements.txt

4 RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

5 COPY ./app /code/app

6 CMD ["fastapi", "run", "app/main.py", "--port", "80"]

```

- 1 Start from the official Python base image.
- 2 Set the current working directory to `/code`.
This is where we'll put the `requirements.txt` file and the `app` directory.
- 3 Copy the file with the requirements to the `/code` directory.
Copy **only** the file with the requirements first, not the rest of the code.
As this file **doesn't change often**, Docker will detect it and use the **cache** for this step, enabling the cache for the next step too.
- 4 Install the package dependencies in the requirements file.
The `--no-cache-dir` option tells `pip` to not save the downloaded packages locally, as that is only if `pip` was going to be run again to install the same packages, but that's not the case when working with containers.

 Note

The `--no-cache-dir` is only related to `pip`, it has nothing to do with Docker or containers.

The `--upgrade` option tells `pip` to upgrade the packages if they are already installed.

Because the previous step copying the file could be detected by the **Docker cache**, this step will also **use the Docker cache** when available.

Using the cache in this step will **save** you a lot of **time** when building the image again and again during development, instead of **downloading and installing** all the dependencies **every time**.

- 5 Copy the `./app` directory inside the `/code` directory.

As this has all the code which is what **changes most frequently** the Docker **cache** won't be used for this or any **following steps** easily.

So, it's important to put this **near the end** of the `Dockerfile`, to optimize the container image build times.

- 6 Set the **command** to use `fastapi run`, which uses Uvicorn underneath.

`CMD` takes a list of strings, each of these strings is what you would type in the command line separated by spaces.

This command will be run from the **current working directory**, the same `/code` directory you set above with `WORKDIR /code`.

 Tip

Review what each line does by clicking each number bubble in the code. 

 Warning

Make sure to **always** use the **exec form** of the `CMD` instruction, as explained below.

Use `CMD` - Exec Form

The `CMD`  Docker instruction can be written using two forms:

 Exec form:

```
#  Do this  
CMD ["fastapi", "run", "app/main.py", "--port", "80"]
```



 Shell form:

```
# 🚫 Don't do this
CMD fastapi run app/main.py --port 80
```

Make sure to always use the **exec** form to ensure that FastAPI can shutdown gracefully and [lifespan events](#) are triggered.

You can read more about it in the [Docker docs for shell and exec form](#).

This can be quite noticeable when using `docker compose`. See this Docker Compose FAQ section for more technical details: [Why do my services take 10 seconds to recreate or stop?](#)

Directory Structure

You should now have a directory structure like:

```
.
├── app
│   ├── __init__.py
│   └── main.py
├── Dockerfile
└── requirements.txt
```

Behind a TLS Termination Proxy

If you are running your container behind a TLS Termination Proxy (load balancer) like Nginx or Traefik, add the option `--proxy-headers`, this will tell Uvicorn (through the FastAPI CLI) to trust the headers sent by that proxy telling it that the application is running behind HTTPS, etc.

```
CMD ["fastapi", "run", "app/main.py", "--proxy-headers", "--port", "80"]
```

Docker Cache

There's an important trick in this `Dockerfile`, we first copy the **file with the dependencies alone**, not the rest of the code. Let me tell you why that is.

```
COPY ./requirements.txt /code/requirements.txt
```

Docker and other tools **build** these container images **incrementally**, adding **one layer on top of the other**, starting from the top of the `Dockerfile` and adding any files created by each of the instructions of the `Dockerfile`.

Docker and similar tools also use an **internal cache** when building the image, if a file hasn't changed since the last time building the container image, then it will **reuse the same layer** created the last time, instead of copying the file again and creating a new layer from scratch.

Just avoiding the copy of files doesn't necessarily improve things too much, but because it used the cache for that step, it can **use the cache for the next step**. For example, it could use

the cache for the instruction that installs dependencies with:

```
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt
```

The file with the package requirements **won't change frequently**. So, by copying only that file, Docker will be able to **use the cache** for that step.

And then, Docker will be able to **use the cache for the next step** that downloads and installs those dependencies. And here's where we **save a lot of time**. ✨ ...and avoid boredom waiting.



Downloading and installing the package dependencies **could take minutes**, but using the **cache** would **take seconds** at most.

And as you would be building the container image again and again during development to check that your code changes are working, there's a lot of accumulated time this would save.

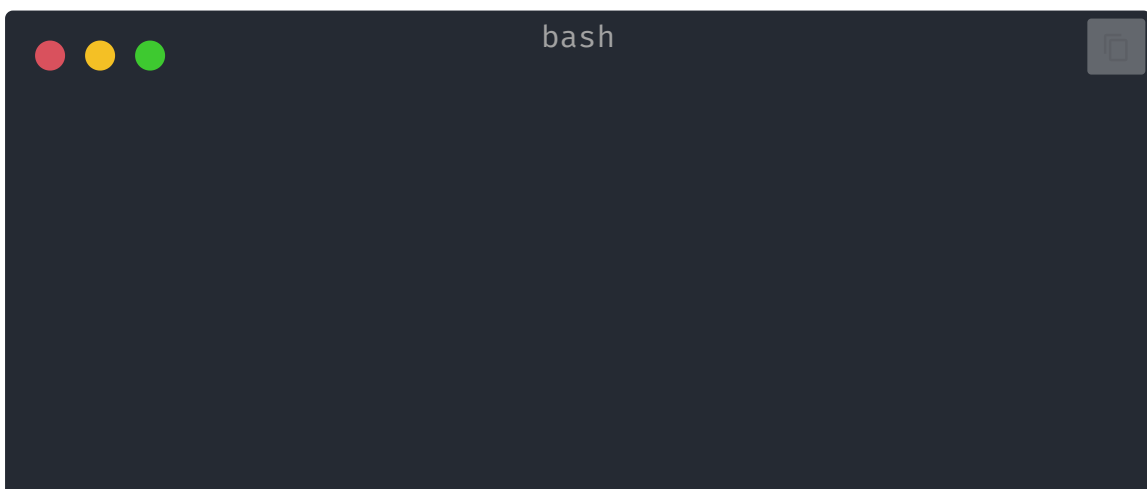
Then, near the end of the `Dockerfile`, we copy all the code. As this is what **changes most frequently**, we put it near the end, because almost always, anything after this step will not be able to use the cache.

```
COPY ./app /code/app
```

Build the Docker Image

Now that all the files are in place, let's build the container image.

- Go to the project directory (in where your `Dockerfile` is, containing your `app` directory).
- Build your FastAPI image:





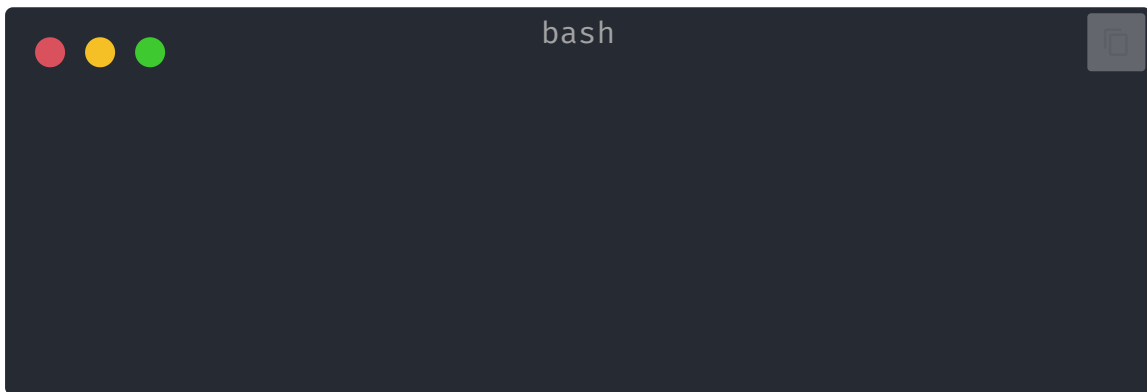
Tip

Notice the `.` at the end, it's equivalent to `./`, it tells Docker the directory to use to build the container image.

In this case, it's the same current directory (`.`).

Start the Docker Container

- Run a container based on your image:



Check it

You should be able to check it in your Docker container's URL, for example:

<http://192.168.99.100/items/5?q=somequery> or <http://127.0.0.1/items/5?q=somequery> (or equivalent, using your Docker host).

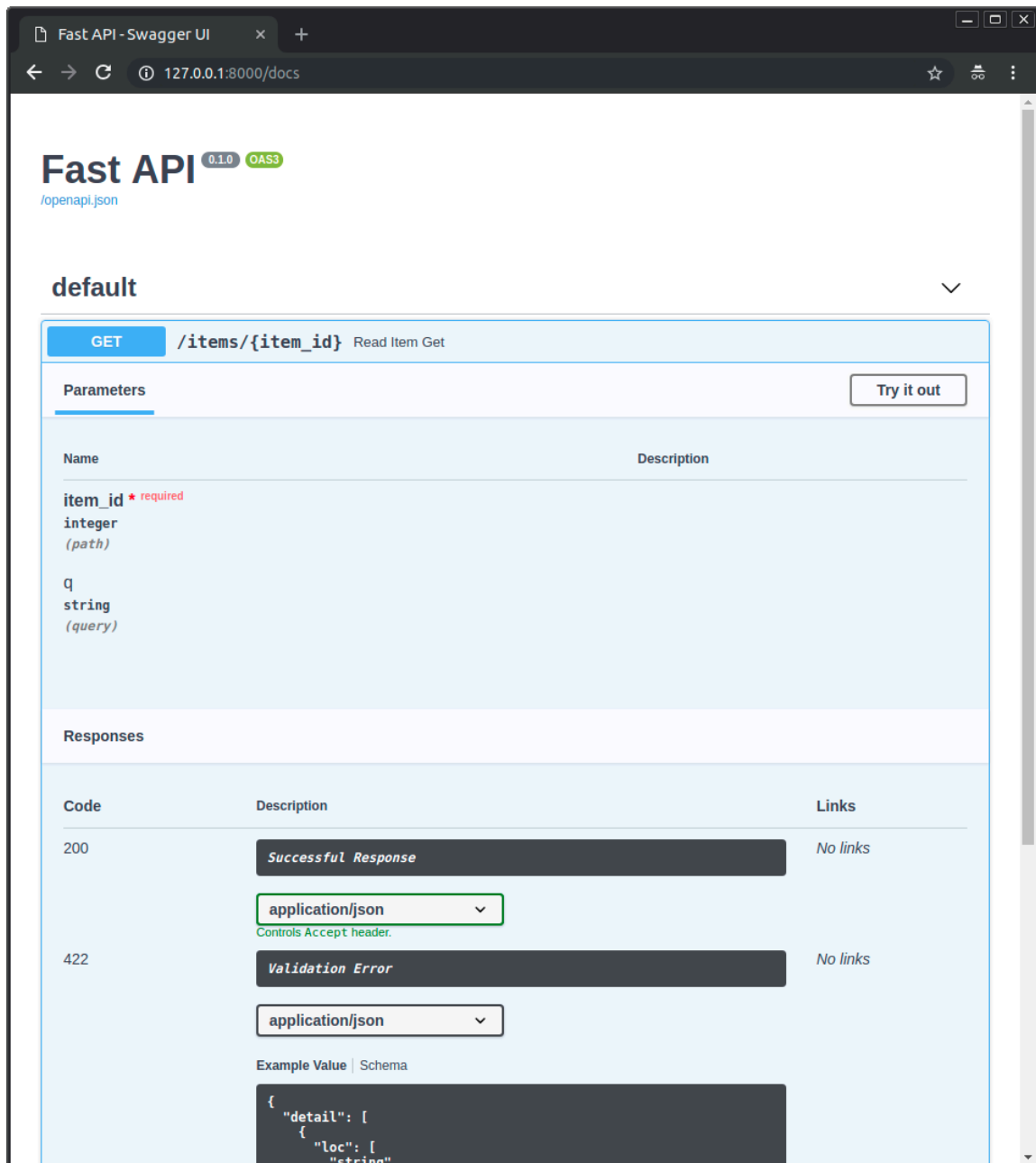
You will see something like:

```
{"item_id": 5, "q": "somequery"}
```

Interactive API docs

Now you can go to <http://192.168.99.100/docs> or <http://127.0.0.1/docs> (or equivalent, using your Docker host).

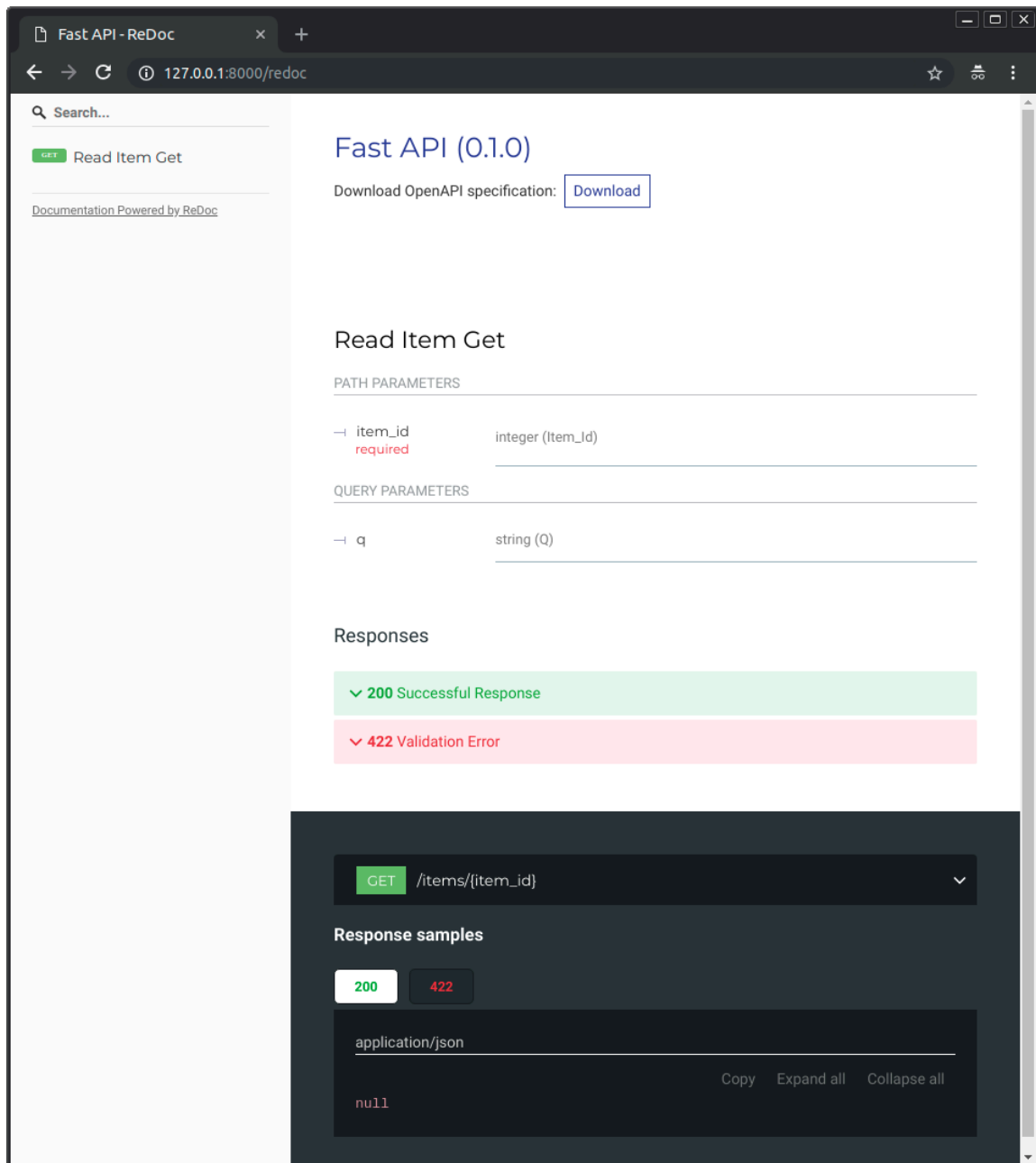
You will see the automatic interactive API documentation (provided by [Swagger UI](#)):



Alternative API docs

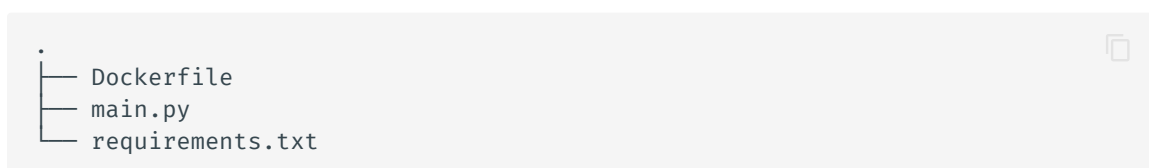
And you can also go to <http://192.168.99.100/redoc> or <http://127.0.0.1/redoc> (or equivalent, using your Docker host).

You will see the alternative automatic documentation (provided by [ReDoc](#)):



Build a Docker Image with a Single-File FastAPI

If your FastAPI is a single file, for example, `main.py` without an `./app` directory, your file structure could look like this:



Then you would just have to change the corresponding paths to copy the file inside the `Dockerfile`:

```
FROM python:3.14

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

1
COPY ./main.py /code/

2
CMD ["fastapi", "run", "main.py", "--port", "80"]
```

- 1 Copy the `main.py` file to the `/code` directory directly (without any `./app` directory).
- 2 Use `fastapi run` to serve your application in the single file `main.py`.

When you pass the file to `fastapi run` it will detect automatically that it is a single file and not part of a package and will know how to import it and serve your FastAPI app. 😎

Deployment Concepts

Let's talk again about some of the same [Deployment Concepts](#) in terms of containers.

Containers are mainly a tool to simplify the process of **building and deploying** an application, but they don't enforce a particular approach to handle these **deployment concepts**, and there are several possible strategies.

The **good news** is that with each different strategy there's a way to cover all of the deployment concepts. 🎉

Let's review these **deployment concepts** in terms of containers:

- HTTPS
- Running on startup
- Restarts
- Replication (the number of processes running)
- Memory
- Previous steps before starting

HTTPS

If we focus just on the **container image** for a FastAPI application (and later the running **container**), HTTPS normally would be handled **externally** by another tool.

It could be another container, for example with [Traefik](#), handling **HTTPS** and **automatic** acquisition of **certificates**.



Tip

Traefik has integrations with Docker, Kubernetes, and others, so it's very easy to set up and configure HTTPS for your containers with it.

Alternatively, HTTPS could be handled by a cloud provider as one of their services (while still running the application in a container).

Running on Startup and Restarts

There is normally another tool in charge of **starting and running** your container.

It could be **Docker** directly, **Docker Compose**, **Kubernetes**, a **cloud service**, etc.

In most (or all) cases, there's a simple option to enable running the container on startup and enabling restarts on failures. For example, in Docker, it's the command line option `--restart`.

Without using containers, making applications run on startup and with restarts can be cumbersome and difficult. But when **working with containers** in most cases that functionality is included by default. ✨

Replication - Number of Processes

If you have a *cluster* of machines with **Kubernetes**, Docker Swarm Mode, Nomad, or another similar complex system to manage distributed containers on multiple machines, then you will probably want to **handle replication** at the **cluster level** instead of using a **process manager** (like Uvicorn with workers) in each container.

One of those distributed container management systems like Kubernetes normally has some integrated way of handling **replication of containers** while still supporting **load balancing** for the incoming requests. All at the **cluster level**.

In those cases, you would probably want to build a **Docker image from scratch** as [explained above](#), installing your dependencies, and running a **single Uvicorn process** instead of using multiple Uvicorn workers.

Load Balancer

When using containers, you would normally have some component **listening on the main port**. It could possibly be another container that is also a **TLS Termination Proxy** to handle **HTTPS** or

some similar tool.

As this component would take the **load** of requests and distribute that among the workers in a (hopefully) **balanced** way, it is also commonly called a **Load Balancer**.



Tip

The same **TLS Termination Proxy** component used for HTTPS would probably also be a **Load Balancer**.

And when working with containers, the same system you use to start and manage them would already have internal tools to transmit the **network communication** (e.g. HTTP requests) from that **load balancer** (that could also be a **TLS Termination Proxy**) to the container(s) with your app.

One Load Balancer - Multiple Worker Containers

When working with **Kubernetes** or similar distributed container management systems, using their internal networking mechanisms would allow the single **load balancer** that is listening on the main **port** to transmit communication (requests) to possibly **multiple containers** running your app.

Each of these containers running your app would normally have **just one process** (e.g. a Uvicorn process running your FastAPI application). They would all be **identical containers**, running the same thing, but each with its own process, memory, etc. That way you would take advantage of **parallelization** in **different cores** of the CPU, or even in **different machines**.

And the distributed container system with the **load balancer** would **distribute the requests** to each one of the containers with your app **in turns**. So, each request could be handled by one of the multiple **replicated containers** running your app.

And normally this **load balancer** would be able to handle requests that go to *other* apps in your cluster (e.g. to a different domain, or under a different URL path prefix), and would transmit that communication to the right containers for *that other* application running in your cluster.

One Process per Container

In this type of scenario, you probably would want to have **a single (Uvicorn) process per container**, as you would already be handling replication at the cluster level.

So, in this case, you **would not** want to have multiple workers in the container, for example with the `--workers` command line option. You would want to have just a **single Uvicorn process** per container (but probably multiple containers).

Having another process manager inside the container (as would be with multiple workers) would only add **unnecessary complexity** that you are most probably already taking care of with your cluster system.

Containers with Multiple Processes and Special Cases

Of course, there are **special cases** where you could want to have a **container** with several **Uvicorn worker processes** inside.

In those cases, you can use the `--workers` command line option to set the number of workers that you want to run:

```
FROM python:3.14

WORKDIR /code

COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

COPY ./app /code/app

1
CMD ["fastapi", "run", "app/main.py", "--port", "80", "--workers", "4"]
```

1 Here we use the `--workers` command line option to set the number of workers to 4.

Here are some examples of when that could make sense:

A Simple App

You could want a process manager in the container if your application is **simple enough** that you can run it on a **single server**, not a cluster.

Docker Compose

You could be deploying to a **single server** (not a cluster) with **Docker Compose**, so you wouldn't have an easy way to manage replication of containers (with Docker Compose) while preserving the shared network and **load balancing**.

Then you could want to have a **single container** with a **process manager** starting **several worker processes** inside.

The main point is, **none** of these are **rules written in stone** that you have to blindly follow. You can use these ideas to **evaluate your own use case** and decide what is the best approach for your system, checking out how to manage the concepts of:

- Security - HTTPS
- Running on startup
- Restarts
- Replication (the number of processes running)
- Memory
- Previous steps before starting

Memory

If you run a **single process per container** you will have a more or less well-defined, stable, and limited amount of memory consumed by each of those containers (more than one if they are replicated).

And then you can set those same memory limits and requirements in your configurations for your container management system (for example in **Kubernetes**). That way it will be able to **replicate the containers** in the **available machines** taking into account the amount of memory needed by them, and the amount available in the machines in the cluster.

If your application is **simple**, this will probably **not be a problem**, and you might not need to specify hard memory limits. But if you are **using a lot of memory** (for example with **machine learning** models), you should check how much memory you are consuming and adjust the **number of containers** that run on **each machine** (and maybe add more machines to your cluster).

If you run **multiple processes per container** you will have to make sure that the number of processes started doesn't **consume more memory** than what is available.

Previous Steps Before Starting and Containers

If you are using containers (e.g. Docker, Kubernetes), then there are two main approaches you can use.

Multiple Containers

If you have **multiple containers**, probably each one running a **single process** (for example, in a **Kubernetes** cluster), then you would probably want to have a **separate container** doing the work of the **previous steps** in a single container, running a single process, **before** running the replicated worker containers.



Note

If you are using Kubernetes, this would probably be an [Init Container](#)

If in your use case there's no problem in running those previous steps **multiple times in parallel** (for example if you are not running database migrations, but just checking if the database is ready yet), then you could also just put them in each container right before starting the main process.

Single Container

If you have a simple setup, with a **single container** that then starts multiple **worker processes** (or also just one process), then you could run those previous steps in the same container, right before starting the process with the app.

Base Docker Image

There used to be an official FastAPI Docker image: [tiangolo/uvicorn-gunicorn-fastapi](#) . But it is now deprecated.

You should probably **not** use this base Docker image (or any other similar one).

If you are using **Kubernetes** (or others) and you are already setting **replication** at the cluster level, with multiple **containers**. In those cases, you are better off **building an image from scratch** as described above: [Build a Docker Image for FastAPI](#).

And if you need to have multiple workers, you can simply use the `--workers` command line option.



Technical Details

The Docker image was created when Uvicorn didn't support managing and restarting dead workers, so it was needed to use Gunicorn with Uvicorn, which added quite some complexity, just to have Gunicorn manage and restart the Uvicorn worker processes.

But now that Uvicorn (and the `fastapi` command) support using `--workers`, there's no reason to use a base Docker image instead of building your own (it's pretty much the same amount of code 😊).

Deploy the Container Image

After having a Container (Docker) Image there are several ways to deploy it.

For example:

- With **Docker Compose** in a single server
- With a **Kubernetes** cluster
- With a Docker Swarm Mode cluster
- With another tool like Nomad
- With a cloud service that takes your container image and deploys it

Docker Image with `uv`

If you are using [uv](#) to install and manage your project, you can follow their [uv Docker guide](#).

Recap

Using container systems (e.g. with **Docker** and **Kubernetes**) it becomes fairly straightforward to handle all the **deployment concepts**:

- HTTPS
- Running on startup
- Restarts
- Replication (the number of processes running)
- Memory
- Previous steps before starting

In most cases, you probably won't want to use any base image, and instead **build a container image from scratch** based on the official Python Docker image.

Taking care of the **order** of instructions in the `Dockerfile` and the **Docker cache** you can **minimize build times**, to maximize your productivity (and avoid boredom). 😎